

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code: CSA12

Course Title: Computer Architecture

**CO Assessed: CO2
Optimisation Task**

Assessment: Assessment Tool 2 – Debugging &

Weightage: 20%

Total Marks: 25

CO2 Statement: Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)

Student Name: RITHISH KUMAR D

Reg. No.: 192565089

Department: CSE-CS

Date:01-08-2026

ANNEXURE A

Question 1

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Binary Analysis

```
+120 = 0 1 1 1 1 0 0 0
+50  = 0 0 1 1 0 0 1 0
-----
Sum   = 1 0 1 0 1 0 1 0    (unsigned value = 170)
```

Interpreted as an 8-bit signed (2's complement) number, the result bit pattern 10101010 has its most significant bit (MSB) set to 1, so it is read as a negative value: -86 . Mathematically, $120 + 50 = 170$, but the maximum value representable in an 8-bit signed (2's complement) format is $+127$, so the true sum of 170 cannot be represented at all — the computed result of -86 is meaningless, confirming that the addition has overflowed.

Overflow Detection

- Sign-based rule: both operands (+120 and +50) are positive (MSB = 0 for each), yet the result's MSB = 1 (negative). Adding two positive numbers can never produce a negative result, so this sign inconsistency by itself proves that overflow has occurred.
- Carry-based rule (used in actual hardware): tracing the addition bit by bit, the carry propagating into the sign bit (bit 7) is 1, while the carry propagating out of the sign bit is 0. Since the carry-in to the MSB (1) differs from the carry-out of the MSB (0), the overflow condition $C_{in} \oplus C_{out} = 1$ is satisfied, confirming overflow through the standard hardware detection mechanism.

Proposed Solution

The Arithmetic Logic Unit (ALU) should implement a dedicated overflow flag (V), computed as the XOR of the carry into the sign bit and the carry out of the sign bit ($V = C_{in} \oplus C_{out}$), or equivalently by checking that the operands share the same sign while the result's sign differs from theirs. Whenever this flag is set after an addition, the system should treat the result as invalid and respond according to the application's needs — for example, raising an overflow interrupt/exception in general-purpose processors, saturating the result to the maximum representable value ($+127$) in signal-processing/DSP systems where saturation arithmetic is preferred, or automatically promoting the operation to a wider data type (e.g., performing the addition in 16-bit instead of 8-bit) when overflow is anticipated, so that the correct value $+170$ can be represented and used safely.

Question 2

A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

Debugging the Bottleneck

In a ripple carry adder, each full-adder stage can only produce its sum and carry-out once it has received the carry-in from the previous (less significant) stage. This means the carry signal must physically "ripple" sequentially through every bit position, from the least significant bit to the most significant bit, before the final, correct result is available. For an n -bit ripple carry adder, the worst-case critical path therefore passes through all n full-adder stages in series, giving a total delay of approximately $n \times$ (delay of one full adder). As the operand width n increases (e.g., 32-bit or 64-bit arithmetic), this delay grows linearly and becomes the dominant bottleneck, which is unacceptable for a real-time system that must complete arithmetic operations within strict timing deadlines.

Optimized Solution — Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder removes the sequential dependency by computing all carry signals in advance, in parallel, rather than waiting for them to ripple through each stage. For each bit position i , two auxiliary signals are defined directly from the inputs: a generate signal $G_i = A_i \cdot B_i$ (a carry is generated at this stage regardless of the incoming carry) and a propagate signal $P_i = A_i \oplus B_i$ (an incoming carry would be passed through this stage). Using these, every carry C_{i+1} can be expressed directly in terms of the original inputs rather than the previous stage's carry, using the recursive expansion $C_{i+1} = G_i + P_i \cdot G_{i-1} + P_i \cdot P_{i-1} \cdot G_{i-2} + \dots + P_i \cdot P_{i-1} \cdot \dots \cdot P_0 \cdot C_0$. Because this expression depends only on the primary inputs (A , B , C_0), all carries can be computed simultaneously by a two-level AND–OR logic network, instead of waiting stage by stage.

How This Reduces Delay

Since every carry bit is derived directly from the generate/propagate signals of the original inputs using a fixed-depth (typically 2-level) logic network, the carry computation delay becomes essentially independent of the operand width n , rather than growing linearly with it as in the ripple carry design. In practice, CLA adders are built in blocks (e.g., 4-bit look-ahead blocks) that are then combined hierarchically for wider operands, giving an overall delay that grows logarithmically with n rather than linearly. This makes the carry look-ahead adder significantly faster for wide operands, at the cost of additional hardware (more gates for the generate/propagate network) — a trade-off that is well justified in a real-time system, where meeting strict timing constraints is more important than the modest increase in circuit area and power.

Question 3

A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

Reviewing Booth's Algorithm

Booth's algorithm multiplies signed numbers by scanning the multiplier's bits two at a time — the current bit Q_n and the bit immediately to its right, $Q_{(n-1)}$, which is initialized to 0 before the first step — and performing one of the following actions on the Accumulator (AC) each cycle: 00 or 11 → no arithmetic operation (only shift); 01 → $AC = AC + \text{Multiplicand}$; 10 → $AC = AC - \text{Multiplicand}$. After the arithmetic step, the combined register (AC, Q, Q-1) is shifted right by one bit, and this repeats for as many cycles as there are bits in the multiplier.

Likely Sources of Error

- a. Using a logical shift instead of an arithmetic shift: after the add/subtract step, the combined (AC, Q, Q-1) register must be shifted right arithmetically — meaning the sign bit (MSB of AC) is preserved and replicated into the vacated top bit — rather than a plain logical shift, which would insert a 0 regardless of sign. When AC holds a negative intermediate value (common whenever the multiplicand or an intermediate partial product is negative), a logical shift corrupts the sign and produces an incorrect result from that point onward.
- b. Missing or incorrect sign extension of the multiplicand: when the multiplicand is loaded into the AC register (which is typically wider than the operand itself, to accommodate the full product), a negative multiplicand must have its sign bit correctly extended to fill the extra bit positions. Loading a negative value without proper sign extension effectively changes its numeric value, corrupting every subsequent add/subtract step.
- c. Incorrect initialization of Q-1: Q-1 must be initialized to 0 before the first cycle; if this is omitted or set incorrectly, the very first bit-pair check (Q_n, Q_{-1}) is evaluated against the wrong reference bit, causing an incorrect operation to be chosen on the first cycle and corrupting the entire computation from the start.
- d. Incorrect number of iterations: the algorithm must run for exactly n cycles for an n -bit multiplier (including the final shift); stopping one cycle early or running one cycle too many produces a partially-shifted, incorrect final product.

Verified Worked Example (Corrected Algorithm)

Multiplying $M = -6$ (4-bit: 1010) by $Q = +3$ (4-bit: 0011), with AC and Q-1 correctly initialized to 0, and using proper arithmetic shifts throughout, gives the following trace:

```
Cycle 1: Qn,Q-1 = 1,0 -> AC = AC - M = 0011 | shift -> AC=0001 Q=1000 Q-1=1
Cycle 2: Qn,Q-1 = 0,1 -> AC = AC + M          | shift -> AC=0001 Q=1100 Q-1=0
Cycle 3: Qn,Q-1 = 0,0 -> no operation          | shift -> AC=1101 Q=1100 Q-1=0
Cycle 4: Qn,Q-1 = 0,0 -> no operation          | shift -> AC=1110 Q=1110 Q-1=0
```

Final register (AC, Q) = 1110 1110, which correctly evaluates to -18 in 8-bit two's complement — matching the expected product of $-6 \times 3 = -18$. This confirms that with correct sign extension, correct Q-1 initialization, correct bit-pair checking, and true arithmetic (sign-preserving) shifts at every cycle, Booth's algorithm produces the accurate signed result; any deviation from these four points is the likely source of the incorrect output for negative operands.

Question 4

An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

Debugging the Restoring Division Workflow

In restoring division, each cycle shifts the (Accumulator, Quotient) register pair left by one bit and then unconditionally subtracts the divisor (M) from the accumulator. If the result of this subtraction is negative, it means the divisor did not actually fit at this step, so the algorithm must restore the accumulator by adding the divisor back (undoing the subtraction just performed) and record a quotient bit of 0; if the result is non-negative, no restoration is needed and the quotient bit is recorded as 1. The inefficiency lies precisely in this restoration step: every time a subtraction turns out to be invalid, an entire extra addition operation is performed purely to undo the previous subtraction, contributing nothing toward the actual result. For a quotient with many 0 bits, this means a large fraction of the algorithm's cycles perform two full arithmetic operations (a wasted subtraction followed by a corrective addition) instead of just one, directly increasing execution time — a serious concern in an embedded system where every cycle affects real-time responsiveness.

Optimized Approach — Non-Restoring Division

Non-restoring division eliminates the separate restoration step entirely by using the sign of the previous cycle's result to decide the next cycle's operation, instead of always subtracting and then conditionally correcting: if the previous remainder was non-negative, the current cycle shifts left and subtracts the divisor as usual; if the previous remainder was negative, the current cycle shifts left and adds the divisor instead of subtracting (which has the same net numerical effect as first restoring — i.e., adding M back — and then subtracting M in the next step, but accomplished in a single addition rather than two separate operations). The quotient bit for each cycle is set to 1 if the resulting remainder is non-negative and 0 if it is negative, exactly as before. Only at the very end, if the final remainder happens to be negative, is a single corrective addition of the divisor performed to restore a valid, non-negative final remainder.

Performance Improvement

Because every cycle of non-restoring division performs exactly one addition or subtraction — never both — the algorithm requires, in the worst case, roughly half the number of arithmetic operations compared to restoring division, where negative intermediate results cost two operations (a subtraction plus a restoring addition) instead of one. This directly reduces the total execution time of each division, which is particularly valuable in an embedded system performing frequent division operations under real-time constraints, at only a small added complexity in the control logic needed to decide, based on the sign of the previous result, whether to add or subtract in the current cycle.

Question 5

A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Debugging the Accuracy Issue

- a.** Exponent alignment: IEEE 754 single precision uses 1 sign bit, 8 exponent bits, and 23 mantissa (fraction) bits. Before two floating-point numbers can be added, their exponents must be made equal by right-shifting the mantissa of the number with the smaller exponent by the difference between the two exponents. When one number is very large and the other is very small, this exponent difference can be large — if it exceeds the 23 available mantissa bits, the smaller number's mantissa is shifted completely out of range, meaning it contributes nothing at all to the sum, even though mathematically it should have some (small) effect. This effect, sometimes called "absorption," is a direct source of the inaccuracy observed.
- b.** Normalization: after the aligned mantissas are added, the result may need to be re-normalized (shifted so that it again has a single leading 1 before the binary point, with the exponent adjusted accordingly). This shifting can discard low-order bits that were only partially meaningful after alignment, contributing additional small errors.
- c.** Rounding: the final result must be rounded to fit back into the available 23 mantissa bits (typically using round-to-nearest-even). Each individual rounding step introduces a small error, and when many additions are performed in sequence (e.g., accumulating many small values into a running total), these small rounding errors can accumulate into a significant, visible inaccuracy — this is a well-known issue in scientific/numerical computing.

Optimization Techniques

- i.** Use double precision instead of single precision: IEEE 754 double precision uses 11 exponent bits and 52 mantissa bits, giving both a far larger representable exponent range and dramatically more mantissa precision than single precision's 23 bits. This substantially reduces both the chance that a small number is entirely absorbed during exponent alignment and the magnitude of rounding errors at each step, at the cost of using twice the memory and, on some hardware, somewhat slower arithmetic.
- ii.** Use compensated summation (Kahan summation algorithm): this technique maintains a small running "compensation" variable that captures the low-order bits lost to rounding at each addition step and feeds them back into the next addition, effectively recovering much of the precision that would otherwise be lost — without needing to switch to a wider data type.
- iii.** Reorder the additions (pairwise/hierarchical summation): summing numbers of similar magnitude together first, before combining these partial sums with much larger values, keeps the exponent difference small at each individual addition step, reducing the alignment-related precision loss compared to naively adding a long sequence of values directly into one large accumulator.
- iv.** Use a wider intermediate accumulator: even if the final inputs and outputs must remain single precision, performing the actual accumulation internally in double precision (or higher) and only rounding down to single precision at the very end confines the precision loss to a single final rounding step rather than compounding it across every intermediate addition.

Code of Conduct:

I RITHISH KUMAR.D (192565089) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**MARKS ALLOCATION**

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided